

Welcome back! Here is a clean, step-by-step recap of everything we covered before reaching **Kneser-Ney Smoothing**, organized to help re-anchor your mental models.

1. The Core Problem: Maximum Likelihood Estimation (MLE) & The Zero Trap

- **The Goal:** A language model predicts the next word w_i given preceding context (history) [12–16].
 - **Raw Counts (MLE):** Estimates probabilities using simple relative sample proportions [14–16].
 - **The Flaw:** MLE guarantees the highest possible probability on training data, but assigns **0.0 probability to any unseen word combination**. In downstream applications like speech recognition, a single 0.0 term makes the entire sentence probability zero [19–20].
 - **The Solution: Smoothing**—subtracting probability mass from seen n -grams and reallocating it to unseen or rare ones.
-

2. How We Measure Quality: Cross-Entropy & Perplexity

- **Cross-Entropy (H):** Measures the average number of bits required to encode each word in test text using the model. Lower cross-entropy means better prediction and compression [34–35].
 - **Perplexity ($PP = 2^H$):** Measures the model's **effective branching factor** or average uncertainty per word [34, 320–321]. A perplexity of 100 means the model is as uncertain as choosing uniformly at random among 100 candidate words.
-

3. The Two Big Paradigms: Backoff vs. Interpolation

When higher-order counts (n -grams) are sparse or zero, models fall back on lower-order distributions ($(n - 1)$ -grams) [48–49, 86–89]:

- **Interpolation (Always Blend):** Always combines predictions from higher-order and lower-order models using a weighted average [87–88].
- **Backoff (Conditional Fallback):** Uses **only** the higher-order model when an n -gram has been observed; falls back to the lower-order model **strictly when the higher-order count is zero** [86–87].

4. The 4 Stepping-Stone Algorithms Leading Up to Kneser-Ney

1. Jelinek-Mercer (Linear Interpolation)

- **Idea:** Blends distributions using weights λ (e.g., $P = \lambda P_{\text{bigram}} + (1 - \lambda) P_{\text{unigram}}$).
- **Flaw:** Uses **linear scaling**, which over-discounts high counts and under-discounts low counts [224–225].

2. Katz Smoothing (Canonical Backoff)

- **Idea:** Uses **Good-Turing estimation** to discount small counts ($r \leq 5$) and **backs off** to unigrams for zero counts [57, 59–60, 86–87].
- **Flaw:** Because it relies on strict backoff, it ignores useful lower-order evidence when low non-zero counts are present [229–230].

3. Witten-Bell Smoothing (Dynamic Interpolation)

- **Idea:** Sets interpolation weight λ dynamically based on **vocabulary diversity** (N_{1+} : how many distinct unique words have ever followed that history) [64–66].
- **Flaw:** Tends to overestimate novel word probabilities on sparse text datasets [37–38, 163–171].

4. Absolute Discounting (Constant Subtraction D)

- **Idea:** Instead of multiplying counts by λ , subtract a **fixed constant** D (e.g., 0.5 or 0.75) from every observed count [71–73].
- **Why it worked:** Empirical analysis showed that ideal count reductions across real language corpora are roughly flat/constant rather than proportional [73, 222–223].

5. The Wall We Hit (The Bridge to Kneser-Ney)

Absolute Discounting introduced the correct formula for subtracting mass, but it suffered from **one major flaw**:

- Its lower-order term relies on **raw unigram frequencies** [74–76].
- This leads to the **"San Francisco" problem**: A word like *"Francisco"* occurs frequently in text, but almost exclusively after *"San"* [75–76]. When Absolute Discounting falls back to unigrams for an unseen context (like *"read ____"*), raw unigram frequency assigns a high probability to *"read Francisco"* simply because *"Francisco"* was frequent overall in a completely unrelated context [75–76].

This brings us right back to **Kneser-Ney Smoothing**, which replaces raw unigram frequencies with **continuation probabilities** [74–77].